

Reading Material 3:

Common String Operations in C

(Learn by reading code: representation, library functions, and common pitfalls)

1. What is a String in C?

C does **not** have a built-in string type.

A string is simply:

A sequence of characters stored in a `char` array, terminated by `'\0'` (null character).

```
#include <stdio.h>

int main() {
    char name[] = "IIT ISM";
    printf("%s\n", name);
}
```

Memory layout:

`'I' 'I' 'T' ' ' 'I' 'S' 'M' '\0'`

Takeaway:

- Strings are character arrays.
- The null character `'\0'` marks the end.
- Without `'\0'`, it is not a valid C string.

2. Declaring Strings

Method 1: Array initialization

```
char s1[] = "Hello";
```

Compiler automatically adds `'\0'`.

Method 2: Manual initialization

```
char s2[] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

Method 3: Fixed size

```
char s3[10] = "Hi";
```

Remaining elements are filled with `'\0'`.

⚠ Important Difference:

```
char s[] = "Hello";      // modifiable
char *p = "Hello";      // string literal (read-only memory)
```

Modifying `p[0]` causes undefined behavior.

Takeaway:

- Use `char array[]` if you need to modify the string.
 - String literals should not be modified.
-

3. Input and Output of Strings

```
#include <stdio.h>

int main() {
    char name[50];

    printf("Enter name: ");
    scanf("%s", name);    // stops at space

    printf("Hello %s\n", name);
}
```

Problem:

- `scanf("%s")` stops at whitespace.

Better alternative:

```
fgets(name, sizeof(name), stdin);
```

Takeaway:

- Prefer `fgets()` for safe input.
 - Always provide buffer size.
-

4. Standard String Library (`<string.h>`)

Include:

```
#include <string.h>
```

Common functions:

- `strlen()` → length
 - `strcpy()` → copy
 - `strcat()` → concatenate
 - `strcmp()` → compare
 - `strchr()` → find character
 - `strstr()` → find substring
-

5. `strlen()`

```
char s[] = "Hello";  
printf("%zu\n", strlen(s)); // 5
```

Important:

- `strlen()` does NOT count `'\0'`.
 - Runs until it finds null character.
-

6. `strcpy()` and `strcat()`

Copy

```
char src[] = "IIT";  
char dest[10];  
  
strcpy(dest, src);
```

Concatenate

```
char s[20] = "IIT ";  
strcat(s, "Dhanbad");
```

⚠ Danger:

- Destination must have enough space.
- No automatic bounds checking.

Safer alternative (modern C):

```
strncpy(dest, src, sizeof(dest));
```

Takeaway:

- Always ensure destination buffer is large enough.
 - Buffer overflow is a serious security issue.
-

7. strcmp()

```
char a[] = "apple";  
char b[] = "banana";  
  
int result = strcmp(a, b);
```

Return value:

- 0 → equal
- <0 → a comes before b
- >0 → a comes after b

Important:

- It compares lexicographically (ASCII order).
 - Case-sensitive.
-

8. Manual String Traversal

Strings are arrays → you can loop over them.

```
char s[] = "Hello";  
  
for (int i = 0; s[i] != '\0'; i++) {  
    printf("%c ", s[i]);  
}
```

Alternative (pointer style):

```
char *p = s;  
  
while (*p != '\0') {  
    printf("%c ", *p);  
    p++;  
}
```

Takeaway:

- `s[i]` and `*(s+i)` are equivalent.
 - Strings end when `'\0'` is reached.
-

9. Common Pitfalls

1 Missing Null Terminator

```
char s[5] = {'H','e','l','l','o'}; // ❌ No '\0'
printf("%s\n", s); // Undefined behavior
```

2 Buffer Overflow

```
char s[5];
strcpy(s, "Hello"); // 6 characters including '\0'
```

Overflow occurs.

3 Comparing Strings Using ==

```
if (a == b) // ❌ compares addresses, not content
```

Correct:

```
if (strcmp(a, b) == 0)
```

4 Modifying String Literal

```
char *p = "Hello";
p[0] = 'h'; // ❌ Undefined behavior
```

10. Example Programs

Example 1: Reverse a String

```
#include <stdio.h>
#include <string.h>

int main() {
    char s[] = "IIT";
    int len = strlen(s);

    for (int i = len - 1; i >= 0; i--) {
        printf("%c", s[i]);
    }
}
```

Example 2: Count Vowels

```
#include <stdio.h>

int main() {
    char s[] = "IIT ISM Dhanbad";
    int count = 0;

    for (int i = 0; s[i] != '\0'; i++) {
        char c = s[i];
        if (c=='a' || c=='e' || c=='i' || c=='o' || c=='u' ||
            c=='A' || c=='E' || c=='I' || c=='O' || c=='U')
            count++;
    }

    printf("Vowels: %d\n", count);
}
```

11. String vs Character Array Size

```
char s1[] = "Hello";
char s2[20] = "Hello";

printf("%zu\n", sizeof(s1)); // 6
printf("%zu\n", sizeof(s2)); // 20
```

Takeaway:

- `sizeof()` gives total allocated size.
 - `strlen()` gives logical length.
-

12. Key Rules to Remember

- A C string is a null-terminated char array.
 - Always allocate extra space for `'\0'`.
 - Never use `==` to compare strings.
 - Ensure buffer is large enough before copying.
 - Prefer `fgets()` over `scanf("%s")`.
 - Strings are arrays — you can traverse using index or pointer.
-